

Módulo de Remarketing v2

Sistema de retención automatizado · enrola.shop

16 reglas activas

15 detectores de signals

5 canales

A/B + geo overrides

Documento técnico y operacional

Generado: abril 2026

Stack: Medusa v2 · PostHog · PostgreSQL · Resend · WhatsApp · Telegram

Tabla de contenidos

1. Resumen ejecutivo	
2. Arquitectura del sistema	
2.1 Diagrama de flujo end-to-end	
2.2 Stack tecnológico	
2.3 Componentes principales	
3. Esquema de base de datos	
3.1 Tabla remarketing_rule	
3.2 Tabla remarketing_fire	
3.3 Tabla legacy remarketing_log (mantenida)	
4. Catálogo de detectores de signals (15)	
5. Catálogo de reglas (16)	
6. Sistemas transversales	
6.1 Dedup unificado cross-tabla	
6.2 A/B testing con variants	
6.3 Geo overrides por ciudad	
6.4 Atribución de conversión last-touch	
6.5 Captura PostHog server-side	
7. Canales de dispatch (5)	
8. UI de administración (4 sub-pestañas)	
9. Operación y monitoreo	
10. Pendientes de instrumentación storefront	
11. Apéndices	
A. Cron jobs activos	
B. Variables de entorno	
C. Endpoints API admin	
D. Mapeo legacy ↔ v2	

1. Resumen ejecutivo

El **Módulo de Remarketing v2** es un sistema de retención automatizado que detecta señales de comportamiento, intención y ciclo del cliente, y dispara mensajes personalizados (email, WhatsApp o alerta interna a Telegram) con guardas estrictas de cooldown, capacidad diaria y horarios silencio.

Reemplaza al sistema viejo basado en jobs por campaña (que sigue corriendo en paralelo para casos específicos) por un **motor de reglas** que toma decisiones por usuario individual a partir de:

- Datos transaccionales de Medusa (órdenes, customer, ciudad)
- Eventos de comportamiento de PostHog (carrito, checkout, búsquedas, scroll)
- Estado del catálogo (productos nuevos, recién restockeados)
- Datos de loyalty (puntos acumulados)

Métricas clave del sistema

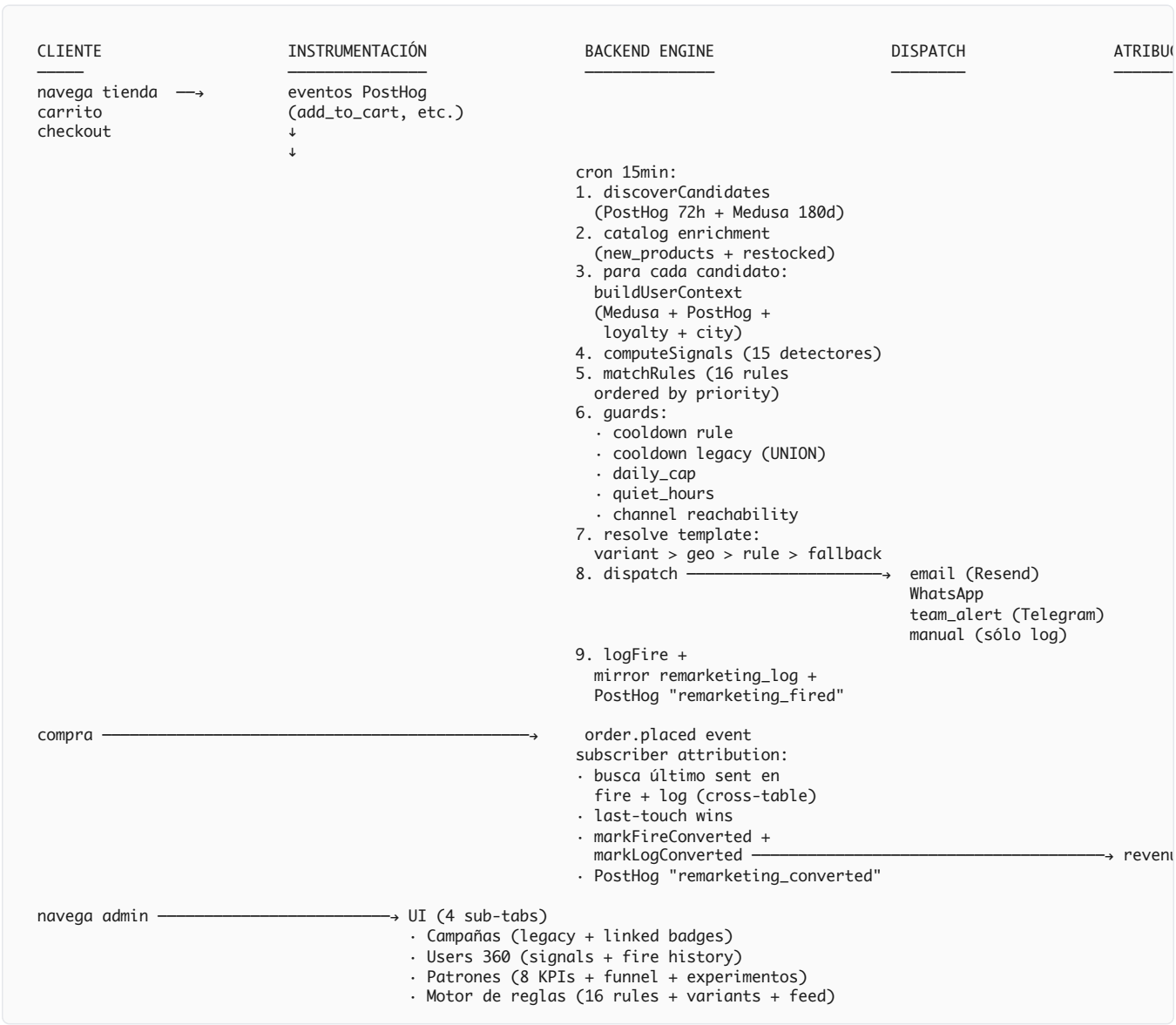
Componente	Cantidad	Estado
Detectores de signals	15	producción
Reglas activas	7 (de 16)	producción
Reglas dormant	9	listas para activar
Canales de dispatch	5	email, WA, auto, team_alert, manual
Cron principal	cada 15 min	activo
Dedup cross-tabla	UNION old + new	activo
Cooldown por regla	48h–720h	configurable
Quiet hours VE	22-9 hr	configurable

Ventajas frente al sistema viejo

- **Personalización por usuario**, no por cohort masivo
- **Cooldown granular por regla** + dedup cross-tabla automático
- **Atribución de revenue** por last-touch entre engine y jobs legacy
- **A/B testing** nativo con hash determinístico
- **Localización** por ciudad (3 ciudades configuradas + extensible)
- **Auditabilidad completa**: cada fire registra context, severity, variant, ciudad
- **Compatibilidad bidireccional** con el sistema viejo (no hay duplicación de envíos)

2. Arquitectura del sistema

2.1 Flujo end-to-end



2.2 Stack tecnológico

Capa	Tecnología	Rol
Backend framework	Medusa v2.13	Server, jobs, subscribers, admin extensions
Storefront	Next.js 16 App Router	Captura de eventos cliente
Base de datos	PostgreSQL 16	<code>remarketing_rule</code> , <code>remarketing_fire</code> , <code>remarketing_log</code>
Behavior analytics	PostHog Cloud (US)	Eventos, persons, HogQL queries
Email	Resend API	Despacho transaccional
WhatsApp	Meta Cloud API + bot custom	Despacho personalizado
Team alerts	Telegram Bot API	Notificaciones internas
Hosting	VPS Hostinger 72.60.114.242	Docker Compose
Cron	Medusa scheduled jobs	Engine cada 15 min + jobs legacy

2.3 Componentes principales

Engine (`remarketing-engine.ts`)

Orquesta candidates → context → signals → rules → guards → dispatch. Cron `*/15 * * * *` .

Detectores (`remarketing-signals.ts`)

15 funciones puras (UserContext → Signal[]). Cada una identifica una oportunidad específica.

UserContext builder (`remarketing-user-context.ts`)

Une Medusa customer/orders + PostHog person/events/behavior + loyalty + city.

Rules DB layer (`remarketing-rules-db.ts`)

CRUD reglas + fires + stats por regla y por variant. Cooldown SQL queries.

Patterns (`remarketing-patterns.ts`)

Agregación cross-usuario para vista bird's-eye. Funnel + experiments + friction.

Dedup unificado (`remarketing-db.ts`)

`alreadyNotified()` hace UNION sobre log + fire. Single source of truth para "ya le mandamos algo".

Attribution subscriber (`remarketing-attribution.ts`)

Listener de `order.placed` . Cross-table last-touch.

Team alert (`team-alert.ts`)

Helper Telegram para canal `team_alert` .

3. Esquema de base de datos

3.1 Tabla remarketing_rule

Una fila por regla del engine. Toda la configuración runtime está aquí.

Columna	Tipo	Propósito
key	VARCHAR(120) PK	Identificador estable (ej. cart_abandoned_v2)
name	VARCHAR(255)	Etiqueta humana
description	TEXT	Documentación visible al operador
enabled	BOOLEAN	Toggle on/off
priority	INT	Mayor prioridad ejecuta primero (anti-dupe interno)
cooldown_hours	INT	Mínimo entre fires de esta regla a un usuario
channel	VARCHAR(20)	email · whatsapp · auto · team_alert · manual
match_signal_id	VARCHAR(120)	Match exacto del signal.id
match_signal_prefix	VARCHAR(120)	Match por prefijo (ej. multi_view_no_cart:)
min_severity	VARCHAR(20)	Umbral de severity: high/medium/low/info
email_subject_template	TEXT	Subject con placeholders {first_name} , etc.
email_html_template	TEXT	HTML body opcional
whatsapp_template	TEXT	Texto WA
quiet_hours_start / quiet_hours_end	INT (0-23)	Ventana de silencio en hora VE
daily_cap	INT	Tope de fires por día por regla
variants	JSONB	A/B variants — array {key, weight, templates}
geo_overrides	JSONB	Overrides por ciudad { city: { templates } }
created_at / updated_at	TIMESTAMPTZ	Auditoría

3.2 Tabla remarketing_fire

Auditoría completa: una fila por cada (regla × usuario × intento de dispatch). Retiene historial 60+ días para análisis y atribución.

Columna	Tipo	Propósito
id	UUID PK	Identificador del fire
rule_key	VARCHAR(120)	FK lógica a remarketing_rule.key
customer_id · email · phone · distinct_id	VARCHAR	Identificadores del usuario
signal_id	VARCHAR(255)	El signal específico que disparó (puede contener :product_id)
signal_severity	VARCHAR(20)	Severidad capturada al fire
channel	VARCHAR(20)	Canal resuelto al momento
status	VARCHAR(40)	pending · sent · failed · skipped_cooldown · skipped_quiet_hours · skipped_cap · skipped_no_channel · dry_run · converted
subject / body	TEXT	Mensaje renderizado (post-template substitution)
error_message	TEXT	Razón de fallo si aplica
fired_at	TIMESTAMPTZ	Momento del intento
converted_order_id	VARCHAR(120)	Set por subscriber attribution
converted_at	TIMESTAMPTZ	Momento de la conversión atribuida
context	JSONB	signal_context, rendered, variant, city, geo_override_applied, legacy_type

Índices

- idx_fire_rule_email_time (rule_key, email, fired_at DESC) — cooldown lookups
- idx_fire_rule_customer_time (rule_key, customer_id, fired_at DESC)
- idx_fire_status_time (status, fired_at DESC) — feed listings
- idx_fire_fired_at (fired_at DESC) — pagination

3.3 Tabla legacy remarketing_log (mantenida)

Tabla del sistema viejo. Sigue siendo escrita por jobs legacy **y también por el engine** (mirror) para que la timeline sea única y el dedup funcione cross-sistema.

Columna	Tipo	Propósito
id	SERIAL PK	Auto-increment
type	VARCHAR	abandoned_cart, birthday, win_back, restock, post_purchase, friction, multi_view_no_cart, fulfillment_delay, ...
recipient_email	VARCHAR	Destinatario
reference_id	VARCHAR	cart_id u order_id según el caso
subject	TEXT	Asunto enviado
metadata	JSONB	cedula, fire_id (mirror), source='engine' 'legacy', converted_order_id (post-attr)
sent_at	TIMESTAMPTZ	Momento del envío

4. Catálogo de detectores de signals

Cada detector es una función pura (ctx: UserContext) => Signal[] . No hace I/O, no escribe DB. computeSignals() los corre todos y devuelve la lista ordenada por severity.

#	ID del signal	Severity	Disparador	Canal sugerido
1	abandoned_checkout	high	checkout_started 1-72h sin order_placed + email identificado	WA si mobile+IG, email
2	cart_abandoned	medium	add_to_cart 2-168h sin order_placed	email
3	multi_view_no_cart:<pid>	medium	3+ product_viewed en 14d sin add_to_cart de ese producto	email
4	restock_due:<pid>	low/medium	2+ órdenes del mismo producto + ciclo medio cumplido (±3d gracia)	email
5	high_engagement_no_convert	high	3+ sesiones, 15+ pageviews, 0 órdenes	manual
6	win_back	medium/high	Sin orden 60+ días	WA si visitó <14d, email si no
7	vip	info	LTV ≥ \$100 (signal informativo)	email
8	coupon_failed_no_purchase	high	coupon_failed 30min-24h sin order_placed	WA si mobile+phone, email
9	multiple_abandoned_checkouts	high	2-20 checkout_started en 14d sin order_placed	manual → team_alert
10	vip_early_access	medium	VIP + catalog.new_products last 7d	email
11	loyalty_redemption_nudge	low	Loyalty balance ≥ 500 + sin orden 30d	email
12	out_of_stock_back:<pid>	high	out_of_stock_view 30d + producto en catalog.recently_restocked	email
13	variant_explorer:<pid>	medium	5+ variant_selected mismo producto en 24h sin add_to_cart	WhatsApp humano
14	search_no_result_followup	medium	2+ search_performed sin click ni cart en 30min, 7d window	email
15	cart_value_decreased	low	product_removed_from_cart 1-48h sin add_to_cart recovery	email gentil
F1	weekly_visitor_dropped	low	4+ sesiones, age ≥28d, ahora 14-60d sin visita	email gentil
F2	post_discount_review:<oid>	low	Orden con discount_code + 7-14d después	email
F3	geo_cluster:<city>	info	Customer en ciudad activa (informativo, no fire)	manual / batch
F5	repurchase_referral	low	3+ órdenes + última hace 5-15d	email

Composición de signals: el mismo usuario puede emitir múltiples signals en un mismo tick. El engine procesa todas en orden de severity, pero impone firedRuleKeys para evitar que la misma regla dispare dos veces. vip + birthday componen vía sinergia D3 en el job legacy.

5. Catálogo de reglas

16 reglas configuradas. Cada una mapea uno o más signals a un canal y un mensaje. Las marcadas **activa** ya están corriendo en el cron de 15 min; las **dormant** esperan que el operador las active desde el panel.

Regla	Estado	Canal	Pri	Cooldown	Cap/día	Match signal
abandoned_checkout_v2	activa	auto	100	48h	40	abandoned_checkout
coupon_failed_no_purchase	activa	auto	95	168h	30	coupon_failed_no_purchase
multiple_abandoned_checkouts	activa	team_alert	90	336h	10	multiple_abandoned_checkouts
out_of_stock_waitlist	dormant	email	85	720h	100	prefix out_of_stock_back:
cart_abandoned_v2	activa	email	80	72h	30	cart_abandoned
vip_early_access	dormant	email	70	720h	50	vip_early_access
variant_explorer_indecision	dormant	whatsapp	65	168h	15	prefix variant_explorer:
restock_due_v2	activa	email	60	168h	50	prefix restock_due:
multi_view_no_cart_v2	activa	email	50	168h	20	prefix multi_view_no_cart:
search_no_result_followup	dormant	email	45	336h	20	search_no_result_followup
win_back_v2	activa	auto	40	720h	15	win_back
repurchase_referral	dormant	email	35	720h	15	repurchase_referral
loyalty_redemption_nudge	dormant	email	30	720h	20	loyalty_redemption_nudge
post_discount_review	dormant	email	30	720h	20	prefix post_discount_review:
weekly_visitor_dropped	dormant	email	25	720h	15	weekly_visitor_dropped
cart_value_decreased	dormant	email	20	168h	20	cart_value_decreased

6. Sistemas transversales

6.1 Dedup unificado cross-tabla

El gran cambio operacional del Batch C1: **el sistema viejo y el nuevo nunca se pisan**.

Antes había riesgo de doble disparo:

- Cron viejo `abandoned-cart` (cada 2h) escribía en `remarketing_log`
- Cron nuevo `remarketing-engine` (cada 15min) escribía en `remarketing_fire`
- Ninguno consultaba la otra tabla → mismo usuario podía recibir 2 emails en minutos

Después de C1, `alreadyNotified()` en `remarketing-db.ts` hace UNION:

```
// Source 1: remarketing_log (legacy)
SELECT 1 FROM remarketing_log
WHERE type = $1
      AND sent_at > NOW() - INTERVAL '1 hour' * $2
      AND (lower(recipient_email) = $email OR metadata->>'cedula' = $cedula)

UNION ALL

// Source 2: remarketing_fire (engine)
SELECT 1 FROM remarketing_fire
WHERE rule_key = ANY($mapped_rule_keys)
      AND status IN ('sent', 'pending', 'converted')
      AND fired_at > NOW() - INTERVAL '1 hour' * $2
      AND (lower(email) = $email OR customer_id = $customerId)
```

Y al revés, cada send exitoso del engine se **espeja** a `remarketing_log` para que jobs legacy lo vean inmediatamente.

Mapping de reglas → buckets legacy

rule_key (engine)	legacy_type bucket
cart_abandoned_v2	abandoned_cart
abandoned_checkout_v2	
coupon_failed_no_purchase	
win_back_v2	win_back
restock_due_v2	restock
multi_view_no_cart_v2	multi_view_no_cart
friction_*	friction

6.2 A/B testing con variants

Cada regla puede definir un campo `variants` JSONB con N variantes. El engine elige cuál usar por usuario con un hash determinístico (mismo usuario siempre cae en la misma variante dentro de la regla).

Schema de variants

```
[
  {
    "key": "A",
    "weight": 50,
    "whatsapp_template": "Hola {first_name}, te ayudo?"
  },
  {
    "key": "B",
    "weight": 50,
    "whatsapp_template": "🔥 Hey {first_name}! Tu {product} te espera."
  }
]
```

Picker (DJB2 hash)

```
function pickVariant(variants, identity) {
  const totalWeight = variants.reduce((s, v) => s + v.weight, 0)
  const bucket = djb2Hash(`${rule.key}:${identity}`) % totalWeight
  let cum = 0
  for (const v of variants) {
    cum += v.weight
    if (bucket < cum) return v
  }
}
```

Stats por variante (30d)

Vista en sub-tab Patrones → "📌 Experimentos A/B activos". Calcula:

- Fires por variante
- Sent / converted / conversion rate por variante
- Líder (variant con mayor conv_rate)
- Lift relativo (líder vs runner-up)
- Significancia básica (≥ 30 sent + lift $\geq 20\%$)

6.3 Geo overrides por ciudad

Cada regla puede definir overrides de templates por ciudad de envío. La ciudad se resuelve en `buildUserContext` consultando la última dirección de envío del cliente.

Schema

```
{
  "cumaná": { "whatsapp_template": "🌴 Cumaná: ..." },
  "caracas": { "whatsapp_template": "🏢 Caracas: MRW al estudio" },
  "maracaibo": { "whatsapp_template": "🌞 Maracaibo: Zoom Express" }
}
```

Precedencia de templates

variant > geo override > rule template > signal fallback

Variants son la capa **experimental** (A/B). Geo es la capa de **localización**. Si una variante define template, gana — así un A/B test no se contamina por la ciudad.

6.4 Atribución de conversión last-touch

Subscriber `order.placed` ejecuta dos queries paralelas:

1. Última fila `remarketing_fire` con status='sent' del usuario (30d)
2. Última fila `remarketing_log` NO mirroreada (metadata.source ≠ 'engine') del usuario (30d)

Compara timestamps, atribuye al más reciente (last-touch). Si fire side gana, llama `markFireConverted` . Si log side gana, llama `markLogConverted` (estampa metadata.converted_order_id en la fila legacy).

Filtro crítico: `remarketing_log` excluye entries que el engine espejó (esos ya están atribuibles via fire). Evita doble counting.

6.5 Captura PostHog server-side

Dos eventos custom server-side se capturan:

Evento	Cuándo	Propiedades
<code>remarketing_fired</code>	Cada send exitoso del engine	rule_key, signal_id, signal_severity, channel, variant, fire_id, has_email, has_phone, legacy_type
<code>remarketing_converted</code>	Cada atribución del subscriber	rule_key, signal_id, channel, variant, fire_id, order_id, hours_to_convert, attributed_via

Permite construir funnels en PostHog que correlacionan signal → fire → conversión sin tocar la base de datos local.

7. Canales de dispatch

Canal	Implementación	Uso típico
email	<code>sendEmail()</code> via Resend API	Mensajes formales, broad reach
whatsapp	<code>sendRemarketingWhatsApp()</code> via Meta Cloud API	Alta intención + mobile + IG, rescates
auto	Resolver: WA si hay phone + signal sugiere WA, email en otro caso	Default razonable para la mayoría de reglas
team_alert	<code>sendTeamAlert()</code> via Telegram Bot	Lead caliente que requiere humano (no spam al cliente)
manual	Sólo log, ningún dispatch	Marcar visualmente sin enviar (signal informativo)

Reglas de resolución (channel = "auto")

- Si `signal.suggested_channel === "whatsapp"` AND tiene phone → **whatsapp**
- Si tiene email → **email**
- Si tiene phone (sin email) → **whatsapp**
- Sin phone ni email → **skip dispatch** (status = `skipped_no_channel`)

Anti-spam por canal

- WhatsApp**: requiere phone válido, sigue el rate limit del bot Meta + cooldown rule + dedup cross-tabla
- Email**: Resend API + rate limit del dominio + cooldown rule + dedup cross-tabla
- Team alert**: cooldown más amplio (336h–720h) — el equipo no quiere ruido
- Todos respetan **quiet hours** (default 22–9 hora VE, configurable por regla)

8. UI de administración

4 sub-pestañas dentro de  **Remarketing** en el admin Medusa:

8.1 Campañas (legacy)

- Cards de campañas viejas: welcome, abandoned_cart, birthday, win_back, post_purchase, restock, pending_payment, stockout_alert, graduation
- Toggle on/off + edición de discount_code, hours_delay, geo_overrides legacy
- **Badge de vinculación:** muestra qué reglas v2 mapean a esta campaña + estado de cada una
- Botón "Editar →" salta a sub-tab Motor de reglas
- Texto explicativo: "Apagar esta campaña no apaga las reglas v2"

8.2 Users 360

- Buscador por email / customer_id / distinct_id (autodetecta)
- Header del usuario: nombre, email, phone, ciudad, fecha desde, botones rápidos (WA, email, PostHog ↗, Clarity ↗)
-  **Señales activas:** cards con severity coloreada + mensaje sugerido + botón "Enviar WA / Email" con texto pre-cargado
- **Grid 8 KPIs:** órdenes, LTV, AOV, días desde última, sesiones, pageviews, vistas de producto, add-to-carts
- **Top productos vistos y top páginas**
-  **Historial de remarketing:** lista de fires con status badges + count enviados/convertidos. Click expande mensaje completo + error + atribución
-  **Órdenes recientes** con items + monto + fecha
-  **Timeline de eventos** (últimos 50 PostHog events)

8.3 Patrones (agregado)

- **8 KPIs activos:** checkouts/carritos/multi-view/restock-due/win-back-elegibles/VIPs/usuarios 72h/ingresos atribuidos
-  **Funnel 7d:** candidatos → señales → fires → enviados → convertidos + conversion rate badge
-  **Mix de canales 7d:** barras email vs WhatsApp vs manual
-  **Actividad 7d:** barras verticales por día con tooltip (sent/conv/failed/skipped)
-  **Productos con fricción:** top 10 productos vistos 3+ veces sin add_to_cart + botón "+ Regla" (crea regla auto-generada para ese producto específico)
-  **Productos con clientes en ventana de reorden:** ciclo medio detectado + count de clientes en ventana
-  **Experimentos A/B activos:** por cada regla con variants, barras horizontales con conv_rate, badge líder + lift% + significancia
-  **Rendimiento por regla 30d:** tabla con fires, enviados, convertidos, conv%, ingresos atribuidos

8.4 Motor de reglas

- **Panel superior:** contador "X/Y reglas activas · cron 15 min" + botones  **Dry-run** y  **Ejecutar ahora**
- **Lista de 16 reglas** como cards expandibles:
 - Header: switch on/off, nombre, badge match_signal, badge canal, badge prioridad,  **Campaña vinculada (estado)**
 - Stats inline: cooldown, cap, fires 30d, conv%, ingresos atrib.
 - Click "Editar" expande formulario completo:

- Nombre, descripción
 - Canal (5 opciones)
 - Severidad mínima, prioridad, cooldown, daily_cap
 - Quiet hours start/end
 - Templates email subject + email HTML + WhatsApp con placeholders
 -  **A/B Variants:** textarea JSON con validación en vivo + tabla de stats por variant
 -  **Geo overrides:** textarea JSON por ciudad con validación de campos permitidos
-  **Feed de dispatches 7d:** tabla filtrable por regla y status (9 estados). Cada fila clickeable expande detalle (asunto, mensaje completo, error, atribución, severity, identificadores)

9. Operación y monitoreo

9.1 Cómo activar una regla dormant

1. Admin →  Remarketing →  Motor de reglas
2. Ubica la regla (las dormant tienen switch off)
3. Click **Editar** y revisa templates, cooldown, daily_cap
4. Click **Guardar cambios** si modificaste algo
5. Activa el switch on/off — toast confirmará
6. Próximo tick del cron (≤ 15 min) la regla empieza a evaluar candidatos

9.2 Cómo verificar que una regla dispara correctamente

1. Click  **Dry-run** en el panel superior
2. Toast muestra: candidatos, signals, sent (en dry-run no envía pero cuenta), skipped, failed
3. Click  **Ejecutar ahora** para forzar un tick real (sin esperar 15 min)
4. Baja al feed → debería aparecer la fila correspondiente
5. Click la fila → revisa subject, body, status final

9.3 Cómo investigar un usuario específico

1. Admin →  Remarketing →  Users 360
2. Busca por email o customer_id o distinct_id
3. Revisa **Señales activas** para ver qué oportunidades hay
4. Revisa **Historial de remarketing** para ver qué se le envió y si convirtió
5. Click un fire para ver el mensaje exacto y la atribución

9.4 Cómo crear una regla A/B

1. Edita una regla existente (o crea una nueva)
2. En el campo  **A/B Variants**, pega JSON array con 2+ variantes
3. Cada variante necesita `key` y `weight` ; opcionalmente sus templates
4. Validación en vivo te dice si el JSON está bien
5. Guarda → próximos fires se reparten por hash determinístico
6. En 1-2 semanas con tráfico real, mira la tabla de  Rendimiento por variante" en el editor o el widget Experimentos en Patrones

9.5 Estados posibles de un fire

Status	Significado
pending	Logueado, aún sin dispatch resolver — transitorio
sent	Dispatch exitoso (email/WA/team_alert OK)
failed	Dispatch falló — ver error_message
skipped_cooldown	Regla o cross-table dedup activo
skipped_quiet_hours	Hora local en ventana de silencio
skipped_cap	Daily cap alcanzado para hoy
skipped_no_channel	Canal incompatible con datos del usuario
dry_run	Modo dry-run, no se envió
converted	Subscriber attribution lo marcó tras una orden

10. Pendientes de instrumentación

Reglas dormant que esperan una pequeña adición en el storefront para activarse con datos reales:

Regla	Falta	Donde
out_of_stock_waitlist	Llamar trackOutOfStockView({product_id, title, handle}) cuando PDP carga con stock=0	storefront/src/app/productos/[handle]/...

Eventos que ya están instrumentados y no requieren trabajo adicional:

- coupon_failed — checkout page ✓
- variant_selected — ProductActions ✓
- search_performed — SearchBar ✓
- product_removed_from_cart — CartDrawer + carrito page ✓
- add_to_cart , checkout_started , order_placed , product_viewed , \$pageview ✓

11. Apéndices

A. Cron jobs activos

Job	Schedule	Sistema	Función
remarketing-engine	* /15 * * * *	v2	Engine principal: candidates → signals → rules
abandoned-cart	0 * /2 * * *	legacy	Recovery basado en Medusa cart entity (mantenido)
birthday-emails	0 9 * * *	legacy + sinergia D3	Cumpleaños + VIP boost via signals
post-purchase	0 11 * * *	legacy	Email N días post-compra
restock	0 12 * * *	legacy	Productos configurados con días explícitos
pending-payment	0 * * * *	legacy	Recordatorio de pago
stockout-alert	30 9 * * *	legacy	Alerta admin de stock bajo
graduation	0 10 * * *	legacy	Manual sale → web (cedula match)
fulfillment-delay	0 * /2 * * *	v2	E4: apología proactiva 24h+ pendiente
win-back	0 10 * * 1	deprecated	Reemplazado por win_back_v2 (settings disabled)

B. Variables de entorno relevantes

Variable	Uso
DATABASE_URL	Conexión PostgreSQL (compartida)
POSTHOG_HOST	https://us.i.posthog.com
POSTHOG_PROJECT_ID	ID del proyecto PostHog
POSTHOG_PERSONAL_API_KEY	Personal API key (server-side queries HogQL)
POSTHOG_PROJECT_API_KEY	Public project key (server-side captures)
RESEND_API_KEY	Email dispatch
RESEND_FROM	Sender de emails
STORE_URL	URL del storefront para CTAs en emails
ADMIN_URL	URL del admin para deeplinks en team_alert

C. Endpoints API admin

Método	Path	Función
GET	/admin/remarketing	Stats legacy + settings + products
POST	/admin/remarketing	Update setting / save crosssell / save restock rules / test email
GET	/admin/remarketing/rules	Lista reglas con stats + variant_stats
POST	/admin/remarketing/rules	Upsert regla
POST	/admin/remarketing/rules?action=toggle	On/off de una regla
POST	/admin/remarketing/rules?action=delete	Eliminar regla
GET	/admin/remarketing/fires	Feed paginado con filtros
POST	/admin/remarketing/engine/run	Trigger manual (dry_run o real)
GET	/admin/remarketing/patterns	Resumen agregado para sub-tab Patrones
GET	/admin/remarketing/user360	Vista 360° por email/customer_id/distinct_id

D. Mapeo legacy ↔ v2 (resumen)

Campaña legacy	Reglas v2 vinculadas	Política
abandoned_cart	cart_abandoned_v2 , abandoned_checkout_v2 , coupon_failed_no_purchase	Dual-stack con dedup cross-table (C1)
birthday	(sin regla v2)	Job legacy enriquecido con signals (sinergia D3 — VIP boost)
win_back	win_back_v2	Legacy deprecated , v2 lo reemplaza
restock	restock_due_v2	Complementarios: legacy = productos configurados, v2 = ciclo detectado por usuario
post_purchase	(sin regla v2)	Job legacy mantenido
welcome · pending_payment · stockout_alert · graduation	(sin regla v2)	Jobs legacy mantenidos sin overlap